交易控制 1：使用 BEGIN TRAN / COMMIT / ROLLBACK

此範例示範：

- 建立訂單（插入 Order）
- 扣減庫存（更新 Goods.stock）
- 若購買數量 > 庫存 → 自動 ROLLBACK

應用情境：會員 M001 想要購買商品 G001(目前庫存:1)，數量為 5。

```sql
BEGIN TRY
    -- 開始交易
    BEGIN TRANSACTION;

    -- 宣告變數
    DECLARE @BuyQty INT = 5;          -- 購買數量
    DECLARE @TargetMember VARCHAR(20) = 'M001';
    DECLARE @TargetGoods VARCHAR(20) = 'G001';
    DECLARE @NewOrderID VARCHAR(20) = 'O999'; -- 模擬一個新訂單編號

    -- 檢查庫存是否足夠
    DECLARE @CurrentStock INT;
    SELECT @CurrentStock = stock FROM Goods WHERE Goods_ID = @TargetGoods;

    IF @CurrentStock < @BuyQty
    BEGIN
        -- 若庫存不足，主動拋出錯誤，這會跳轉到 CATCH 區塊
        THROW 51000, '庫存不足，無法完成交易！', 1;
    END

    -- 扣除庫存
```

```
交易失敗，已執行 ROLLBACK。
錯誤訊息： 庫存不足，無法完成交易！
```

交易控制 2：不同隔離層級造成查詢行為差異

應用情境：管理者正在後台批次修改價格（正在執行但未 Commit），此時分析師想要查詢目前的商品價格。

```sql
USE OnlineShop;
GO

BEGIN TRANSACTION;

-- 鎖定 G002 商品並將價格暫時改為 9999
UPDATE Goods
SET price = 9999
WHERE Goods_ID = 'G002';

PRINT '管理者已修改價格';
-- 自動還原
WAITFOR DELAY '00:00:15';
ROLLBACK;
PRINT '價格還原。';
```

找不到任何問題

(影響了 1 個資料列)
管理者已修改價格
價格還原。

```sql
USE OnlineShop;
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;

SELECT Goods_ID, price FROM Goods WHERE Goods_ID = 'G002';
PRINT '以上是 READ UNCOMMITTED 的結果';

SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

SELECT Goods_ID, price FROM Goods WHERE Goods_ID = 'G002';
PRINT '以上是 READ COMMITTED 的結果';
```

194 % 找不到任何問題

結果 訊息

	Goods_ID	price
1	G002	9999

	Goods_ID	price
1	G002	243

- 鎖的種類：
  - **排他鎖 (Exclusive Lock, X Lock)**：視窗 A 執行 UPDATE 時，會對 G002 這行資料加上 X 鎖。X 鎖具有排他性，代表「我在改，別人不能改，也不能讀」。
  - **共用鎖 (Shared Lock, S Lock)**：一般的 SELECT 查詢會申請 S 鎖。S 鎖與 X 鎖是不相容的。
- 隔離層級差異：
  - **READ UNCOMMITTED**：查詢時不申請 S 鎖，因此不會被視窗 A 的 X 鎖擋住。

    結果：視窗 B 會看到價格 9999。視窗 A 最後 ROLLBACK ，價格其實是 243。分析師拿到了錯誤的數據。
  - **READ COMMITTED**：查詢時必須申請 S 鎖。因為視窗 A 已經拿著 X 鎖，視窗 B 的 S 鎖申請會進入等待狀態。

結果：視窗 B 會一直轉圈圈，直到視窗 A ROLLBACK 或 COMMIT 釋放 X 鎖後，視窗 B 才會顯示結果 243。

如何避免產生資料不一致問題？

1. 正確使用交易範圍：
   - **原則：**凡是涉及兩個以上資料表異動（如：訂單+庫存、轉帳的扣款+入帳），務必包在 BEGIN TRAN ... COMMIT 中。
   - **縮短交易時間：**交易區塊內的程式碼越少越好，不要在交易中進行網路請求或等待使用者輸入，以免長時間鎖定資料表。
2. 選擇合適的隔離層級：
   - 對於一般查詢，使用預設的 READ COMMITTED 即可避免髒讀。 若是報表統計類查詢，且不在乎些微的時間差誤差，可以使用 READ UNCOMMITTED 避免查詢被交易中的訂單卡住。